

DOCKER – CHALLENGES

1. Create a customized Docker image using a Dockerfile.

create a working directory

- `mkdir docker`
- `cd docker`

2. Create and open Dockerfile

- `vi Dockerfile`

3. Write content inside Dockerfile

You added this configuration:

```
FROM amazonlinux:latest
```

```
RUN yum update -y && \  
    yum install -y nginx && \  
    yum clean all
```

```
RUN mkdir -p /usr/share/nginx/html
```

```
RUN echo "<h1>Hello from Nginx Docker</h1>" > /usr/share/nginx/html/index.html
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

Build the image

- `docker build -t test-nginx .`

```

root@ip-172-31-4-171:~# mkdir docker
root@ip-172-31-4-171:~# ls
alpine-tomcat  docker  docker-compose.yml  multi-stage-example  nginx-task  snap
root@ip-172-31-4-171:~# cd docker
root@ip-172-31-4-171:~/docker# vi Dockerfile
root@ip-172-31-4-171:~/docker# cat Dockerfile
FROM amazonlinux:latest

RUN yum update -y && \
    yum install -y nginx && \
    yum clean all

RUN mkdir -p /usr/share/nginx/html

RUN echo "<h1>Hello from Nginx Docker</h1>" > /usr/share/nginx/html/index.html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

root@ip-172-31-4-171:~/docker# docker build -t test-nginx .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  3.072kB
Step 1/6 : FROM amazonlinux:latest
--> 181f98c48832
Step 2/6 : RUN yum update -y && yum install -y nginx && yum clean all
--> Running in 3100e95bac22
Amazon Linux 2023 repository                122 MB/s | 76 MB    00:00
Last metadata expiration check: 0:00:11 ago on Mon Aug 31 06:49:21 2026.
Dependencies resolved.

```

Run container from image

- `docker run -itd -p 83:80 test-nginx`

```

root@ip-172-31-4-171:~/docker# docker images

IMAGE                                     ID                                DISK USAGE  CONTENT SIZE  EXTRA
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest 55f7b41c5810 318MB       77.8MB
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest 55f7b41c5810 318MB       77.8MB
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:v1      48b6ed03e53f 424MB       121MB        U
alpine-tomcat:latest       0b59e4afe365 465MB       163MB        U
alpine:3.19                6baf43584bcb 11.6MB      3.51MB
amazonlinux:latest         181f98c48832 227MB       59.3MB
custom-nginx:v1            396376a32904 239MB       63.3MB
custom-nginx:v2            4ce855a44999 343MB       94.9MB
eclipse-temurin:8-jre     e8950bfdf2ba 393MB       109MB
jenkins/jenkins:latest    f296d729ad09 707MB       243MB
jenkins/jenkins:lts       8547df3b0db2 801MB       290MB        U
naven-app:latest          b34ddbcfb936 735MB       237MB        U
naven:3.9.6-eclipse-temurin-8 c85a28e9be65 593MB       175MB
multi-satge-app:latest    48b6ed03e53f 424MB       121MB        U
nginx:latest              b34848eff6db 241MB       66.2MB        U
sonarqube:latest          9026624a61cd 2.53GB      1.09GB
srihari0310/amazon-nginx:latest 1ef755255b94 238MB       63.3MB
test-nginx:latest         393e4ac3d39c 318MB       77.8MB
tomcat:latest             b4237a8551b3 589MB       158MB
ubuntu:latest             2260313b31c8 160MB       45.3MB

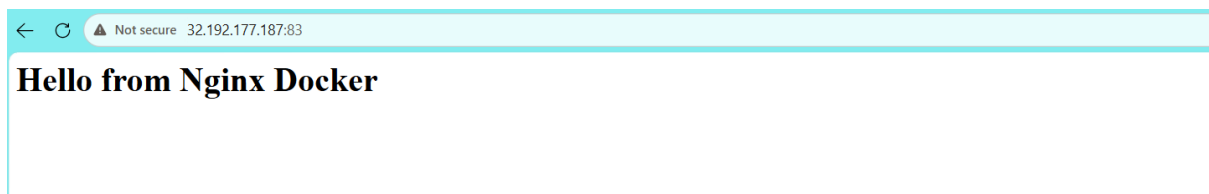
root@ip-172-31-4-171:~/docker# docker run -itd -p 83:80 test-nginx:latest
988328f01798bc7e68e3fa2a955448984cd63a2c6a3adc2439178405cd43cbb0
root@ip-172-31-4-171:~/docker#

```

Access application in browser

Open in browser:

`http://<EC2-Public-IP>:83`



Push the image to Docker Hub.

Login to Docker Hub

- `docker login`

Tag the Local Image

renamed (tagged) your image with your Docker Hub username:

- `docker tag test-nginx srihari0310/test-nginx`

Format:

- `docker tag <local-image> <dockerhub-username>/<repo-name>`

Push Image to Docker Hub

- `docker push srihari0310/test-nginx`

```
root@ip-172-31-4-171:~/docker# docker login
Authenticating with existing credentials... [Username: srihari0310]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
root@ip-172-31-4-171:~/docker# docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:v1	48b6ed03e53f	424MB	121MB	U
alpine-tomcat:latest	0b59e4afe365	465MB	163MB	U
alpine:3.19	6baf43584bcb	11.6MB	3.51MB	
amazonlinux:latest	181f98c48832	227MB	59.3MB	
custom-nginx:v1	396376a32904	239MB	63.3MB	
custom-nginx:v2	4ce855a44999	343MB	94.9MB	
eclipse-temurin:8-jre	e8950bfdf2ba	393MB	109MB	
jenkins/jenkins:latest	f296d729ad09	707MB	243MB	
jenkins/jenkins:lts	8547df3b0db2	801MB	290MB	U
maven-app:latest	b34ddbcbfb936	735MB	237MB	U
maven:3.9.6-eclipse-temurin-8	c85a28e9be65	593MB	175MB	
multi-stage-app:latest	48b6ed03e53f	424MB	121MB	U
nginx:latest	b34848eff6db	241MB	66.2MB	U
sonarqube:latest	9026624a61cd	2.53GB	1.09GB	
srihari0310/amazon-nginx:latest	1ef755255b94	238MB	63.3MB	
test-nginx:latest	393e4ac3d39c	318MB	77.8MB	U

```
root@ip-172-31-4-171:~/docker# docker tag test-nginx:latest srihari0310/test-nginx:latest
root@ip-172-31-4-171:~/docker# docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:v1	48b6ed03e53f	424MB	121MB	U
alpine-tomcat:latest	0b59e4afe365	465MB	163MB	U
alpine:3.19	6baf43584bcb	11.6MB	3.51MB	
amazonlinux:latest	181f98c48832	227MB	59.3MB	
custom-nginx:v1	396376a32904	239MB	63.3MB	
custom-nginx:v2	4ce855a44999	343MB	94.9MB	
eclipse-temurin:8-jre	e8950bfdf2ba	393MB	109MB	
jenkins/jenkins:latest	f296d729ad09	707MB	243MB	
jenkins/jenkins:lts	8547df3b0db2	801MB	290MB	U
maven-app:latest	b34ddbcbfb936	735MB	237MB	U
maven:3.9.6-eclipse-temurin-8	c85a28e9be65	593MB	175MB	
multi-stage-app:latest	48b6ed03e53f	424MB	121MB	U
nginx:latest	b34848eff6db	241MB	66.2MB	U
sonarqube:latest	9026624a61cd	2.53GB	1.09GB	
srihari0310/amazon-nginx:latest	1ef755255b94	238MB	63.3MB	
srihari0310/test-nginx:latest	393e4ac3d39c	318MB	77.8MB	U
test-nginx:latest	393e4ac3d39c	318MB	77.8MB	U
tomcat:latest	b4237a8551b3	589MB	158MB	
ubuntu:latest	2260313b31c8	160MB	45.3MB	

```
root@ip-172-31-4-171:~/docker# docker push srihari0310/test-nginx:latest
The push refers to repository [docker.io/srihari0310/test-nginx]
0fae8e8ec7c7: Pushed
f206cad06171: Mounted from srihari0310/amazon-nginx
a61ea4d4c028: Pushed
latest: digest: sha256:393e4ac3d39c827dac2064491ba1f1dbf04554e50162bf02dcceae6bc21f3e size: 965
root@ip-172-31-4-171:~/docker#
```

Verify on Docker Hub Website

opened Docker Hub and checked:

srihari0310/test-nginx

The screenshot shows the Docker Hub interface for the repository 'srihari0310/test-nginx'. The repository is public and was last pushed 3 minutes ago. The 'General' tab is selected, showing a table of tags. The 'latest' tag is listed with a type of 'Image', pulled 'less than 1 day' ago, and pushed '3 minutes' ago. The interface also includes a sidebar with navigation options like 'Repositories', 'Hardened Images', and 'Collaborations'. On the right, there are sections for 'Docker commands' and 'buildcloud'.

Tag	OS	Type	Pulled	Pushed
latest		Image	less than 1 day	3 minutes

Push the same image to Amazon ECR.

Authenticate Docker with ECR

ran this command:

```
➤ aws ecr get-login-password --region eu-north-1 \
    | docker login --username AWS --password-stdin 885581373585.dkr.ecr.eu-north-1.amazonaws.com
```

This connects Docker to AWS ECR registry.

Tag Image with ECR Repository URL

executed:

```
➤ aws ecr get-login-password --region us-east-1 \
    | docker login --username AWS --password-stdin 299582146630.dkr.ecr.us-east1.amazonaws.com
```

This prepares your image for upload to ECR.

Format:

```
➤ docker tag <local-image>:<tag> <aws-account>.dkr.ecr.<region>.amazonaws.com/<repo>:<tag>
```

Upload Image to ECR

ran:

```
➤ docker push 885581373585.dkr.ecr.eu-north-1.amazonaws.com/amazon-nginx:latest
```

```
root@ip-172-31-4-171:~/docker# aws ecr get-login-password --region us-east-1 \
| docker login --username AWS --password-stdin 299582146630.dkr.ecr.us-east-1.amazonaws.com

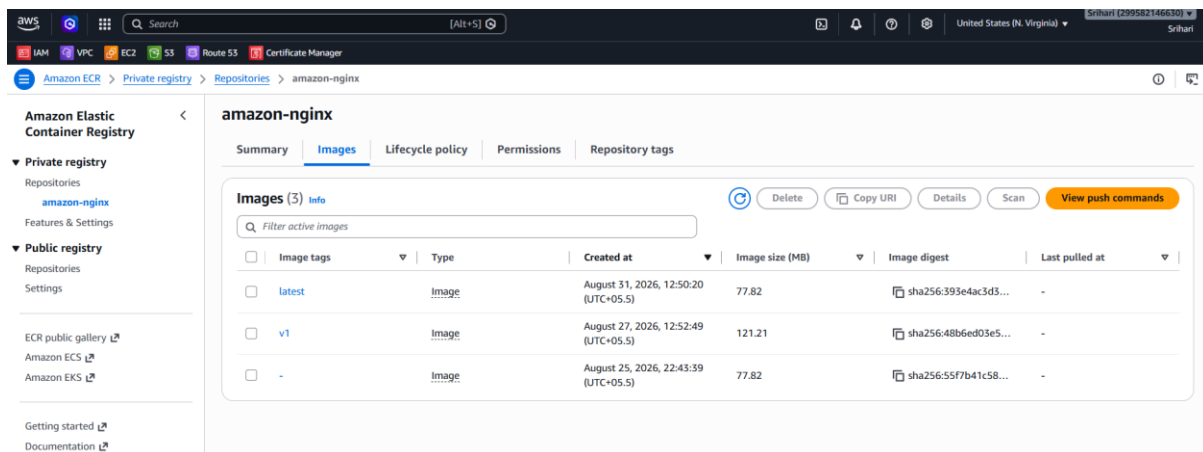
WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
root@ip-172-31-4-171:~/docker# docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:v1	48b6ed03e53f	424MB	121MB	U
alpine-tomcat:latest	0b59e4afe365	465MB	163MB	U
alpine:3.19	6baf43584bcb	11.6MB	3.51MB	
amazonlinux:latest	181f98c48832	227MB	59.3MB	
custom-nginx:v1	396376a32904	239MB	63.3MB	
custom-nginx:v2	4ee859a44999	343MB	94.9MB	
eclipse-temurin:8-jre	e8950bfdf2ba	393MB	109MB	
jenkins/jenkins:latest	f296d729ad09	707MB	243MB	
jenkins/jenkins:lts	8547df3b0db2	801MB	290MB	U
maven-app:latest	b34ddbcbfb936	735MB	237MB	U
maven:3.9.6-eclipse-temurin-8	c85a28e9be65	593MB	175MB	
multi-satge-app:latest	48b6ed03e53f	424MB	121MB	U
nginx:latest	b34848eff6db	241MB	66.2MB	U
sonarqube:latest	9026624a61cd	2.53GB	1.09GB	
srihar:0310/amazon-nginx:latest	1ef755255b94	238MB	63.3MB	
srihar:0310/test-nginx:latest	393e4aac3d39c	318MB	77.8MB	U
test-nginx:latest	393e4aac3d39c	318MB	77.8MB	U
tomcat:latest	b4237a8551b3	589MB	158MB	
ubuntu:latest	2260313b31c8	160MB	45.3MB	

```
root@ip-172-31-4-171:~/docker# docker tag test-nginx:latest 299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest
root@ip-172-31-4-171:~/docker# docker push 299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest
```

Open AWS ECR → Repositories → amazon-nginx → Images



4.Provision one EC2 instance using Terraform and install Jenkins.ile

Install terraform & and extract that file

Configure AWS Credentials

In terminal, ran:

aws configure

And entered:

- Access Key
- Secret Key
- Region: us-east-1
- Format: json

This connects Terraform to AWS.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\SRIHARI\Downloads> terraform --version
Terraform v1.16.0
on windows_amd64
PS C:\Users\SRIHARI\Downloads> terraform init
Terraform initialized in an empty directory!

The directory has no Terraform configuration files. You may begin working
with Terraform immediately by creating Terraform configuration files.
PS C:\Users\SRIHARI\Downloads> pwd

Path
----
C:\Users\SRIHARI\Downloads

PS C:\Users\SRIHARI\Downloads> cd "C:\Users\SRIHARI\OneDrive\Desktop"
PS C:\Users\SRIHARI\OneDrive\Desktop> mkdir terraform-jenkins

Directory: C:\Users\SRIHARI\OneDrive\Desktop

Mode                LastWriteTime         Length Name
----                -
d-----          8/31/2026   2:42 PM             terraform-jenkins

PS C:\Users\SRIHARI\OneDrive\Desktop> cd terraform-jenkins
PS C:\Users\SRIHARI\OneDrive\Desktop\terraform-jenkins> pwd

Path
----
C:\Users\SRIHARI\OneDrive\Desktop\terraform-jenkins

PS C:\Users\SRIHARI\OneDrive\Desktop\terraform-jenkins> notepad main.tf
```

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 6.0"  
    }  
  }  
  
  required_version = ">= 1.5.0"  
}  
  
provider "aws" {  
  region = "us-east-1"  
}  
  
# -----  
# Get Default VPC  
# -----  
  
data "aws_vpc" "default" {  
  default = true  
}  
  
# -----  
# Get Subnets from Default VPC  
# -----  
  
data "aws_subnets" "default" {  
  filter {  
    name = "vpc-id"  
    values = [data.aws_vpc.default.id]  }  
}
```

```
}  
}  
  
# -----  
# Security Group  
# -----  
  
resource "aws_security_group" "jenkins_sg" {  
  name      = "jenkins-security-group"  
  description = "Allow SSH and Jenkins"  
  vpc_id    = data.aws_vpc.default.id  
  
  # SSH - Port 22  
  ingress {  
    description = "SSH"  
    from_port   = 22  
    to_port     = 22  
    protocol    = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
  
  # Jenkins - Port 8080  
  ingress {  
    description = "Jenkins"  
    from_port   = 8080  
    to_port     = 8080  
    protocol    = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
  
  # Allow all outbound traffic
```

```
egress {  
    from_port = 0  
    to_port   = 0  
    protocol  = "-1"  
    cidr_blocks = ["0.0.0.0/0"]  
}
```

```
tags = {  
    Name = "Jenkins-SG"  
}  
}
```

```
# -----  
# EC2 Instance  
# -----
```

```
resource "aws_instance" "jenkins" {
```

```
    # Amazon Linux 2023 AMI - us-east-1
```

```
    ami = "ami-0bdc7d025135d7b49"
```

```
    # Free Tier eligible instance type
```

```
    instance_type = "t3.micro"
```

```
    # AWS Key Pair
```

```
    key_name = "devops.key"
```

```
    # Subnet
```

```
    subnet_id = data.aws_subnets.default.ids[0]
```

```
    # Security Group
```

```
vpc_security_group_ids = [  
    aws_security_group.jenkins_sg.id  
]
```

```
# Assign Public IP
```

```
associate_public_ip_address = true
```

```
# -----
```

```
# Install Java and Jenkins automatically
```

```
# -----
```

```
user_data = <<-EOF
```

```
#!/bin/bash
```

```
# Update packages
```

```
dnf update -y
```

```
# Install Java and curl
```

```
dnf install -y java-21-amazon-corretto curl
```

```
# Add Jenkins repository
```

```
curl -o /etc/yum.repos.d/jenkins.repo \
```

```
https://pkg.jenkins.io/rpm-stable/jenkins.repo
```

```
# Import Jenkins repository key
```

```
rpm --import https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key
```

```
# Install Jenkins
```

```
dnf install -y jenkins
```

```
# Enable Jenkins at boot
```

```
systemctl enable jenkins
```

```
# Start Jenkins
```

```
systemctl start jenkins
```

```
EOF
```

```
tags = {
```

```
    Name = "Jenkins-Server"
```

```
}
```

```
}
```

```
# -----
```

```
# Output Public IP
```

```
# -----
```

```
output "jenkins_public_ip" {
```

```
    value = aws_instance.jenkins.public_ip
```

```
}
```

```
# -----
```

```
# Output Jenkins URL
```

```
# -----
```

```
output "jenkins_url" {
```

```
    value = "http://${aws_instance.jenkins.public_ip}:8080"
```

```
}
```

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

```
# aws_instance.jenkins will be updated in-place
~ resource "aws_instance" "jenkins" {
  id              = "i-06dc259ee780df8d5"
  ~ public_dns    = "ec2-34-237-142-167.compute-1.amazonaws.com" -> (known after apply)
  ~ public_ip     = "34.237.142.167" -> (known after apply)
  tags            = {
    "Name" = "Jenkins-Server"
  }
  ~ user_data     = <<-EOT
    #!/bin/bash

    # Update packages
    dnf update -y

    # Install Java and curl
    dnf install -y java-21-amazon-corretto curl

    # Add Jenkins repository
    curl -o /etc/yum.repos.d/jenkins.repo \
  + https://pkg.jenkins.io/redhat-stable/jenkins.repo
  + https://pkg.jenkins.io/rpm-stable/jenkins.repo
```

```
# aws_instance.jenkins will be created
+ resource "aws_instance" "jenkins" {
+   ami                         = "ami-0c101f26f147fa7fd"
+   arn                         = (known after apply)
+   associate_public_ip_address = (known after apply)
+   availability_zone           = (known after apply)
+   disable_api_stop            = (known after apply)
+   disable_api_termination     = (known after apply)
+   ebs_optimized               = (known after apply)
+   enable_primary_ipv6         = (known after apply)
+   force_destroy               = false
+   get_password_data           = false
```

```
[ec2-user@ip-172-31-6-115 ~]$ java --version
openjdk 21.0.12 2026-07-21 LTS
OpenJDK Runtime Environment Corretto-21.0.12.8.1 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.8.1 (build 21.0.12+8-LTS, mixed mode, sharing)
[ec2-user@ip-172-31-6-115 ~]$ sudo systemctl daemon-reload
[ec2-user@ip-172-31-6-115 ~]$ sudo systemctl restart jenkins
[ec2-user@ip-172-31-6-115 ~]$ sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-08-31 12:41:52 UTC; 10s ago
     Main PID: 26308 (java)
       Tasks: 43 (Limit: 1059)
      Memory: 396.5M
         CPU: 20.852s
    CGroup: /system.slice/jenkins.service
            └─26308 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080
```

Get Public IP from Output

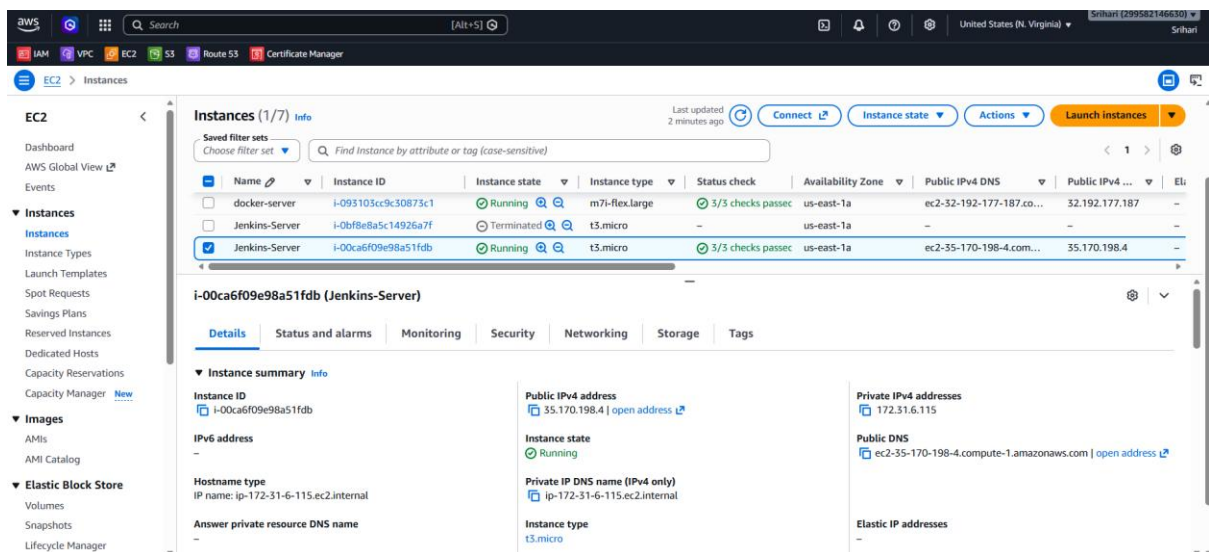
After apply, got:

jenkins_public_ip =35.170.198.4

opened EC2 Dashboard and saw:

- Instance Name: **Jenkins-Server**
- Status: **Running**
- Public IP: http://35.170.198.4:8080

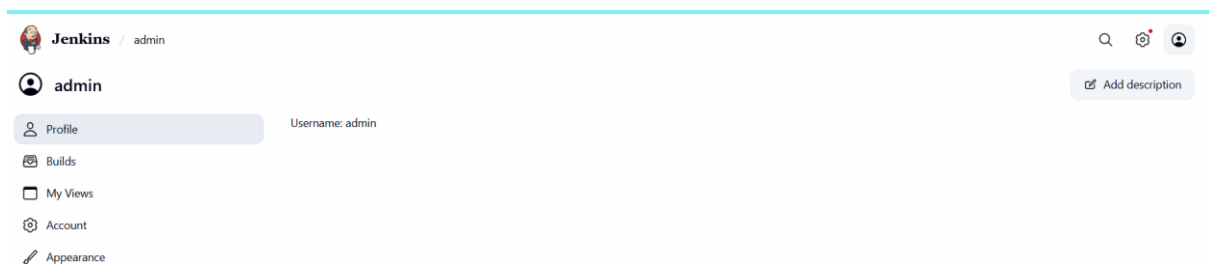
This confirms EC2 was created successfully.



The screenshot displays the AWS Management Console's EC2 Instances page. The 'Jenkins-Server' instance is selected, and its details are shown in the 'Instance summary' section. The instance is in a 'Running' state, using the 't3.micro' instance type, and has a public IP address of 35.170.198.4. The console also shows a list of instances, including 'docker-server' and 'Jenkins-Server'.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...
docker-server	i-093103ec9c30873c1	Running	m7i-flex.large	3/3 checks passed	us-east-1a	ec2-32-192-177-187.co...	32.192.177.187
Jenkins-Server	i-0bfb8e8a5c14926a7f	Terminated	t3.micro	-	us-east-1a	-	-
Jenkins-Server	i-00ca6f09e98a51fdb	Running	t3.micro	3/3 checks passed	us-east-1a	ec2-35-170-198-4.com...	35.170.198.4

➤ Logged into Jenkins through initial admin password



The screenshot shows the Jenkins web interface. The user 'admin' is logged in, and the interface displays a sidebar with navigation links: Profile, Builds, My Views, Account, and Appearance. The main area shows the user's profile and a 'Add description' button.

Create one Jenkins job to build and push the Docker image to Docker Hub.

1. Source: <https://github.com/betawins/Python-app.git>

Install Git

- `yum install git -y`

Used for cloning GitHub repositories.

```
[root@ip-172-31-6-115 ~]# yum install git -y
Last metadata expiration check: 1:19:44 ago on Mon Aug 31 12:38:50 2026.
Dependencies resolved.
```

Package	Architecture	Version	Repository	Size
Installing:				
git	x86_64	2.50.1-1.amzn2023.0.1	amazonlinux	53 k
Installing dependencies:				
git-core	x86_64	2.50.1-1.amzn2023.0.1	amazonlinux	4.9 M
git-core-doc	noarch	2.50.1-1.amzn2023.0.1	amazonlinux	2.8 M
perl-Error	noarch	1:0.17030-2.amzn2023.0.1	amazonlinux	42 k
perl-File-Find	noarch	1.37-477.amzn2023.0.9	amazonlinux	26 k
perl-Git	noarch	2.50.1-1.amzn2023.0.1	amazonlinux	41 k
perl-TermReadKey	x86_64	2.38-9.amzn2023.0.3	amazonlinux	36 k
perl-lib	x86_64	0.65-477.amzn2023.0.9	amazonlinux	15 k
Transaction Summary				
Install 8 Packages				
Total download size: 7.9 M				
Installed size: 41 M				
Downloading Packages:				
(1/8): git-2.50.1-1.amzn2023.0.1.x86_64.rpm			1.3 MB/s 53 kB	00:00
(2/8): git-core-doc-2.50.1-1.amzn2023.0.1.noarch.rpm			41 MB/s 2.8 MB	00:00
(3/8): perl-Error-0.17030-2.amzn2023.0.1.noarch.rpm			1.3 MB/s 42 kB	00:00
(4/8): git-core-2.50.1-1.amzn2023.0.1.x86_64.rpm			47 MB/s 4.9 MB	00:00
(5/8): perl-File-Find-1.37-477.amzn2023.0.9.noarch.rpm			712 kB/s 26 kB	00:00
(6/8): perl-Git-2.50.1-1.amzn2023.0.1.noarch.rpm			1.1 MB/s 41 kB	00:00
(7/8): perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64.rpm			1.3 MB/s 36 kB	00:00
(8/8): perl-lib-0.65-477.amzn2023.0.9.x86_64.rpm			570 kB/s 15 kB	00:00
Total				44 MB/s 7.9 MB 00:00

Install Docker

`yum install docker -y`

Installs Docker engine.

```
[root@ip-172-31-6-115 ~]# yum install docker -y
Last metadata expiration check: 1:20:56 ago on Mon Aug 31 12:38:50 2026.
Dependencies resolved.
```

Package	Architecture	Version	Repository	Size
Installing:				
docker	x86_64	25.0.16-1.amzn2023.0.3	amazonlinux	47 M
Installing dependencies:				
container-selinux	noarch	4:2.245.0-1.amzn2023	amazonlinux	58 k
containerd	x86_64	2.2.5-1.amzn2023.0.1	amazonlinux	24 M
iptables-libs	x86_64	1.8.8-3.amzn2023.0.2	amazonlinux	401 k
iptables-nft	x86_64	1.8.8-3.amzn2023.0.2	amazonlinux	183 k
libgroup	x86_64	3.0-1.amzn2023.0.1	amazonlinux	75 k
libnetfilter_conntrack	x86_64	1.0.8-2.amzn2023.0.2	amazonlinux	58 k
libnftnl	x86_64	1.0.1-19.amzn2023.0.2	amazonlinux	30 k
libnftnl	x86_64	1.2.2-2.amzn2023.0.2	amazonlinux	84 k
pkgconf	x86_64	2.5-1.amzn2023.0.4	amazonlinux	83 k
rsync	x86_64	1.3.5-1.amzn2023.0.2	amazonlinux	3.9 M
Transaction Summary				
Install 11 Packages				
Total download size: 76 M				
Installed size: 284 M				
Downloading Packages:				
(1/11): container-selinux-4.2.245.0-1.amzn2023.noarch.rpm			1.4 MB/s 58 kB	00:00
(2/11): iptables-libs-1.8.8-3.amzn2023.0.2.x86_64.rpm			13 MB/s 401 kB	00:00
(3/11): iptables-nft-1.8.8-3.amzn2023.0.2.x86_64.rpm			6.8 MB/s 183 kB	00:00
(4/11): libgroup-3.0-1.amzn2023.0.1.x86_64.rpm			2.2 MB/s 75 kB	00:00
(5/11): libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64.rpm			963 kB/s 58 kB	00:00
(6/11): libnftnl-1.0.1-19.amzn2023.0.2.x86_64.rpm			1.1 MB/s 30 kB	00:00

By default, Jenkins cannot run Docker commands. So you gave permission.

Add Jenkins User to Docker Group

- `usermod -aG docker jenkins`

```
[root@ip-172-31-6-115 ~]# usermod -aG docker jenkins
[root@ip-172-31-6-115 ~]# systemctl restart docker
[root@ip-172-31-6-115 ~]# systemctl restart jenkins
-bash: systemctl: command not found
[root@ip-172-31-6-115 ~]# systemctl restart jenkins
[root@ip-172-31-6-115 ~]#
```

Open Plugin Manager

Dashboard → Manage Jenkins → Manage Plugins → Available

Search and Install These Plugins

- ✓ Docker
- ✓ Docker Commons
- ✓ Docker Pipeline
- ✓ Docker API
- ✓ Git Server

The screenshot shows the Jenkins 'Manage Jenkins' interface, specifically the 'Plugins' section. On the left, there's a sidebar with 'Available plugins' selected. The main area features a search bar and a table of plugins. The table has columns for 'Install', 'Name', 'Released', and 'Health'. Five plugins are listed: Docker, Docker Commons, Docker Pipeline, Docker API, and Git server. Each plugin entry includes a version number, tags, a description, and a health status indicator (a green circle with a number). The 'Docker Pipeline' plugin has a warning message about its adoption status.

Install	Name	Released	Health
☒	Docker 1327.v9524f1ee134e Cloud Providers Cluster Management docker Provisions Jenkins agents on demand as Docker containers. Runs builds in isolated, disposable environments.	18 days ago	100
☒	Docker Commons 477.v289085a_b_6896 Library plugins (for use by other plugins) docker Provides the common shared functionality for various Docker-related plugins.	1 mo 11 days ago	97
☒	Docker Pipeline 653.v2f2c08eff0ec pipeline DevOps Deployment docker Build and use Docker containers from pipelines. <i>This plugin is up for adoption! We are looking for new maintainers. Visit our Adopt a Plugin initiative for more information.</i>	18 days ago	76
☒	Docker API 3.7.1-136.v5d70f77a_c3d6 Library plugins (for use by other plugins) docker This plugin provides docker-java API for other plugins.	5 mo 1 day ago	100
☒	Git server 137.ve0060b_432302 git Library plugins (for use by other plugins)	1 yr 5 mo ago	94

Open Credentials Page

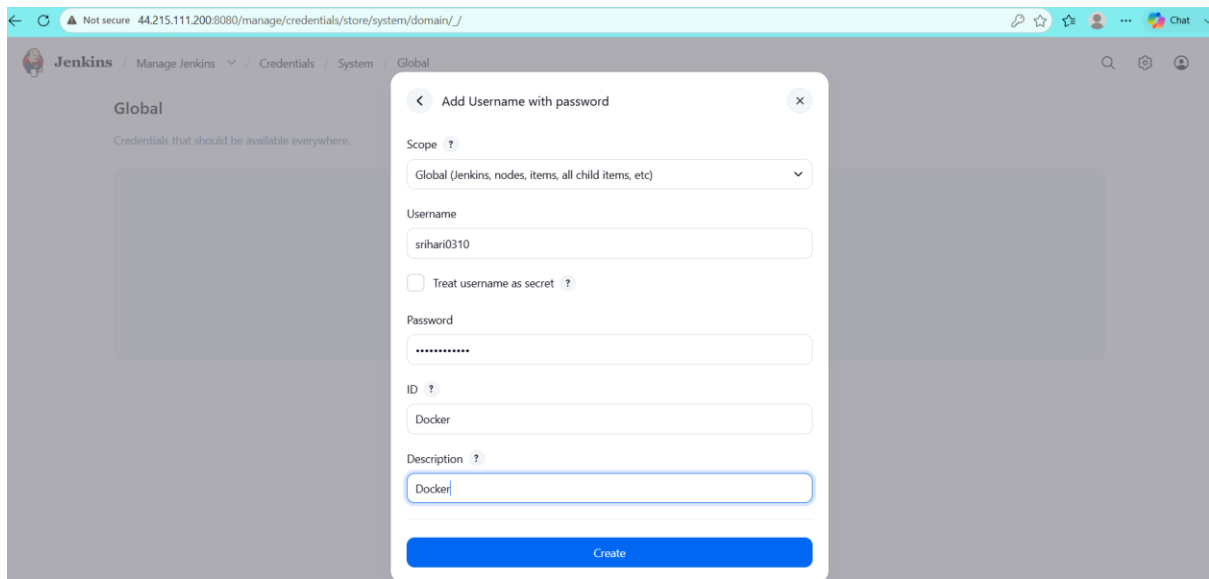
Manage Jenkins → Credentials → System → Global → Add Credentials

Add Credentials

Fill:

Field	Value
Kind	Username & Password
Username	Docker Hub username
Password	Docker Hub password
ID	Docker
Description	Docker

Click **Create**.



Create New Job

Dashboard → New Item

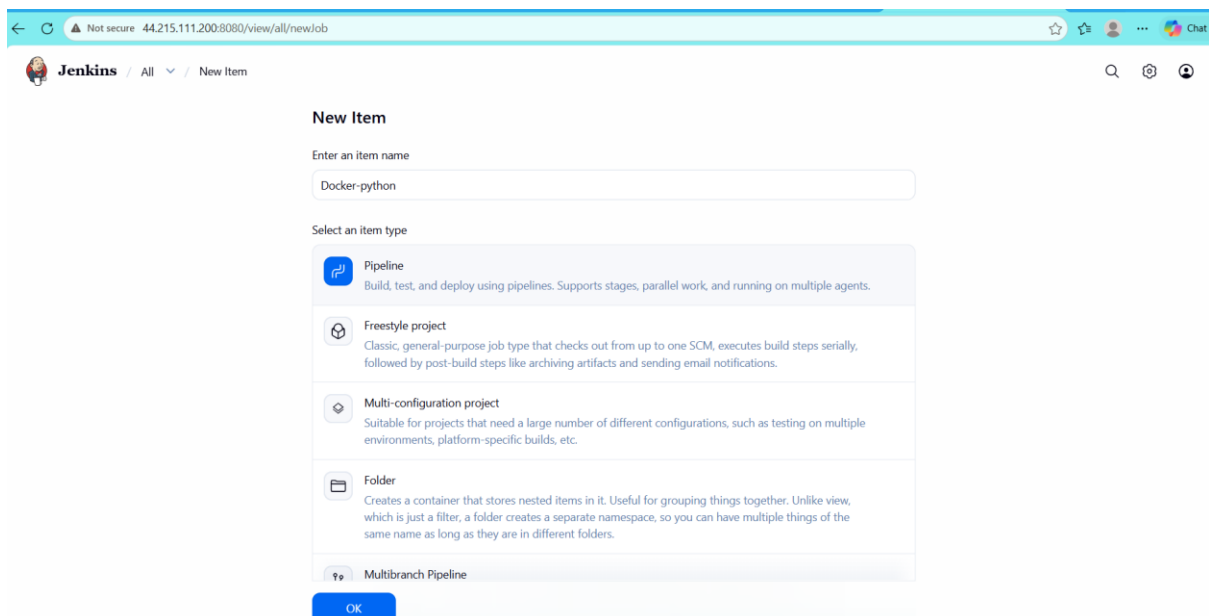
Enter Job Name

Docker-python

Select:

Pipeline

Click **OK**.



```
pipeline {
```

```
    agent any
```

```
    environment {
```

```

IMAGE_NAME = "srihari0310/python-app"

TAG = "v1"
}

stages {
    stage('Checkout Code') {
        steps {
            git branch: 'main', url: 'https://github.com/betawins/Python-app.git'
        }
    }

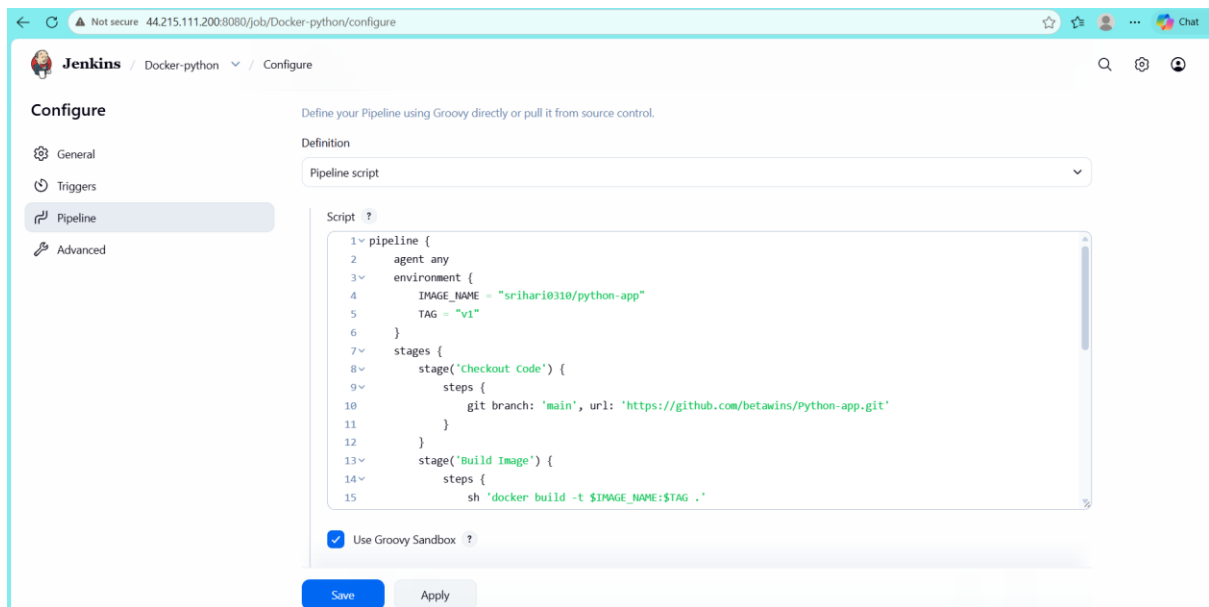
    stage('Build Image') {
        steps {
            sh 'docker build -t $IMAGE_NAME:$TAG .'
        }
    }

    stage('Login to DockerHub') {
        steps {
            withCredentials([usernamePassword(
                credentialsId: 'Docker',
                usernameVariable: 'USER',
                passwordVariable: 'PASS'
            )]) {
                sh 'echo $PASS | docker login -u $USER --password-stdin'
            }
        }
    }

    stage('Push Image') {
        steps {
            sh 'docker push $IMAGE_NAME:$TAG'
        }
    }
}

```

}



trigger Build

Click:

Build Now

14. Monitor Stages

You saw green stages:

- ✓ Checkout Code
- ✓ Build Image
- ✓ Login to DockerHub
- ✓ Push Image

The screenshot shows the Jenkins 'Docker-python' pipeline status page. The 'Status' tab is selected. The pipeline is in a 'Completed' state, indicated by a green checkmark. The 'Stage View' section shows a table of stages and their execution times.

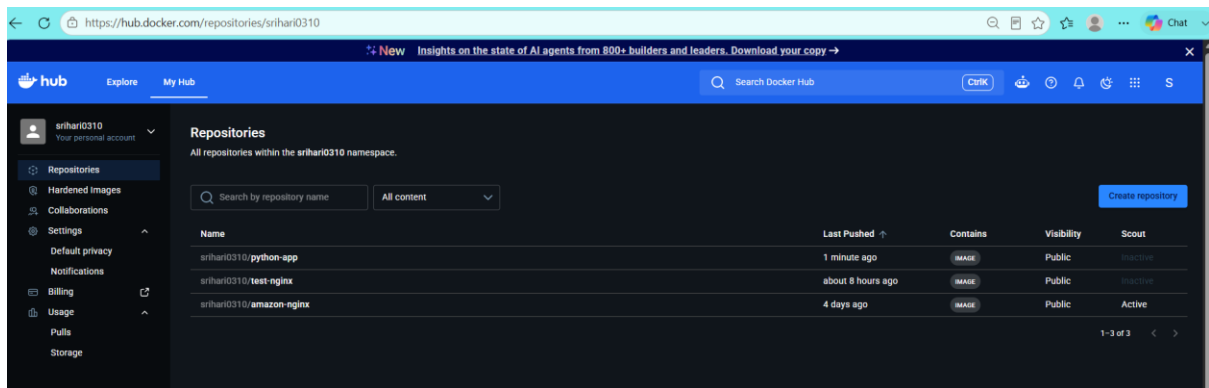
	Checkout Code	Build Image	Login to DockerHub	Push Image
Average stage times: (full run time: ~10min 19s)	4s	28s	809ms	5s
#4 Aug 31 20:17 No Changes	4s	28s	809ms	5s
#3 Aug 31 20:14 No Changes				
#2 Aug 31 20:10 No Changes				
#1 Aug 31 20:07 No Changes				

On the left sidebar, there are links for 'Status', 'Changes', 'Build Now', 'Delete Pipeline', 'Configure', 'Rename', 'Full Stage View', 'Stages', and 'Pipeline Syntax'. At the bottom, there is a 'Builds' section with a filter and a list of builds, including build #4 from 2:47 PM.

Open Docker Hub

logged in and checked:

Repositories → srihari0310/python-app



📁 **Source Codes:** <https://github.com/betawins/docker-tasks.git>

From the **frontend source code**, write a Dockerfile, build a Docker image, run it, and push that image to your Docker registry.

Create Project Folder

- mkdir source-code
- cd source-code

Clone GitHub Repository

- git clone <https://github.com/betawins/docker-tasks.git>

Navigate to Frontend Folder

- cd docker-tasks/Frontend_based_source
- ls

create Dockerfile

- vi Dockerfile

Add Dockerfile Content

FROM nginx:alpine

COPY . /usr/share/nginx/html

EXPOSE 80

CMD ["nginx","-g","daemon off;"]

```
[root@ip-172-31-6-115 ~]# mkdir source-code
[root@ip-172-31-6-115 ~]# ls
source-code
[root@ip-172-31-6-115 ~]# cd source-code/
[root@ip-172-31-6-115 source-code]# git clone https://github.com/betawins/docker-tasks.git
Cloning into 'docker-tasks'...
remote: Enumerating objects: 146, done.
remote: Counting objects: 100% (146/146), done.
remote: Compressing objects: 100% (123/123), done.
remote: Total 146 (delta 12), reused 141 (delta 9), pack-reused 0 (from 0)
Receiving objects: 100% (146/146), 22.11 MiB | 27.05 MiB/s, done.
Resolving deltas: 100% (12/12), done.
[root@ip-172-31-6-115 source-code]# ls
docker-tasks
[root@ip-172-31-6-115 source-code]# cd docker-tasks/
[root@ip-172-31-6-115 docker-tasks]# ls
Frontend_based_source  Java_based_source  NodeJs_based_source  README.md
[root@ip-172-31-6-115 docker-tasks]# cd Frontend_based_source/
[root@ip-172-31-6-115 Frontend_based_source]# ls
index.html  javascript.js  style.css  todayDeal.js
[root@ip-172-31-6-115 Frontend_based_source]# vi Dockerfile
FROM nginx:alpine

COPY . /usr/share/nginx/html

EXPOSE 80

CMD ["nginx","-g","daemon off;"]

[root@ip-172-31-6-115 Frontend_based_source]#
```

Build Image

- `docker build -t srihari0310/frontend-app`

Run Container

- `docker run -d -p 8085:80 --name frontend srihari0310/frontend-app`

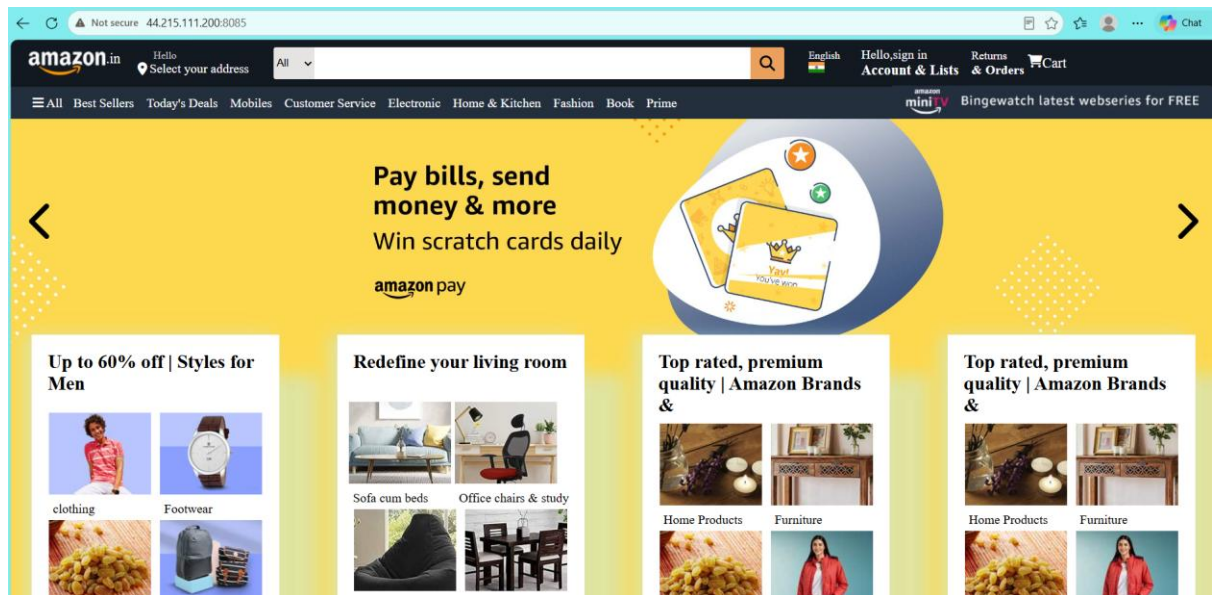
```
[root@ip-172-31-6-115 Frontend_based_source]# docker build -t srihari0310/frontend-app .
[+] Building 2.5s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 191B
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 39.72kB
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:db33bfc0b2951e7f9a72db5db120288c127ffaeb4e6d4b95a26fead017d5913
=> => resolve docker.io/library/nginx:alpine@sha256:db33bfc0b2951e7f9a72db5db120288c127ffaeb4e6d4b95a26fead017d5913
=> => sha256:db33bfc0b2951e7f9a72db5db120288c127ffaeb4e6d4b95a26fead017d5913 10.3kB / 10.3kB
=> => sha256:55afalecc21d2bb5e5045f32dafee56272fcd9860bac26f6c32123439af26a4 3.85MB / 3.85MB
=> => sha256:f2f5fedd5aacc27413031af3a4f8ee4eff0c9d843f6a76e81bfa92245ac5c06 2.50kB / 2.50kB
=> => sha256:7bc3ba2f95da04be123135f45eaf57350672b64eadddcf1169b239f3a53a6e 12.32kB / 12.32kB
=> => sha256:0ea93b727874d2ba2f392aee8dcd3b32c223a12386766810be3bd32c7d 628B / 628B
=> => sha256:1edbe59a7434c6975957bb78530f191c92aeebd9c7309966192ba276a7c0acc1 955B / 955B
=> => extracting sha256:55afalecc21d2bb5e5045f32dafee56272fcd9860bac26f6c32123439af26a4
=> => sha256:d94291c262619626bc7a27b61b090d85365e55c61f92ca49b190a26930121299 1.90MB / 1.90MB
=> => sha256:da4dea1b0aafb3e163ae330a311ecd6978371d99794c8f439b3b609eb9d6d 1.40kB / 1.40kB
=> => sha256:0d62d88506ba1273b40e7c137e3a2c1f5e56370967682820b8c2277a6caeb 404B / 404B
=> => sha256:8cdf4f237782ae36780c4634b1457e8d6f6ef929f3e46b0a2074ba54acd56 1.21kB / 1.21kB
=> => sha256:f597529c91648bab2279939e01b745210875081852c73000aed882c000ff5de 20.50MB / 20.50MB
=> => extracting sha256:d94291c262619626bc7a27b61b090d85365e55c61f92ca49b190a26930121299
=> => extracting sha256:0ea93b727874d2ba2f392aee8dcd3b32c223a12386766810be3bd32c7d
=> => extracting sha256:0d62d88506ba1273b40e7c137e3a2c1f5e56370967682820b8c2277a6caeb
=> => extracting sha256:8cdf4f237782ae36780c4634b1457e8d6f6ef929f3e46b0a2074ba54acd56
=> => extracting sha256:da4dea1b0aafb3e163ae330a311ecd6978371d99794c8f439b3b609eb9d6d
=> => extracting sha256:f597529c91648bab2279939e01b745210875081852c73000aed882c000ff5de

[root@ip-172-31-6-115 Frontend_based_source]# docker run -d -p 8085:80 --name frontend srihari0310/frontend-app
b3956aafd88cc037b430a3ca62406bc98b22daa987452ff27049311524bd010
[root@ip-172-31-6-115 Frontend_based_source]#
```

Access in Browser

Open:

`http://<EC2-Public-IP>:8085`



login to Docker Hub

- docker login

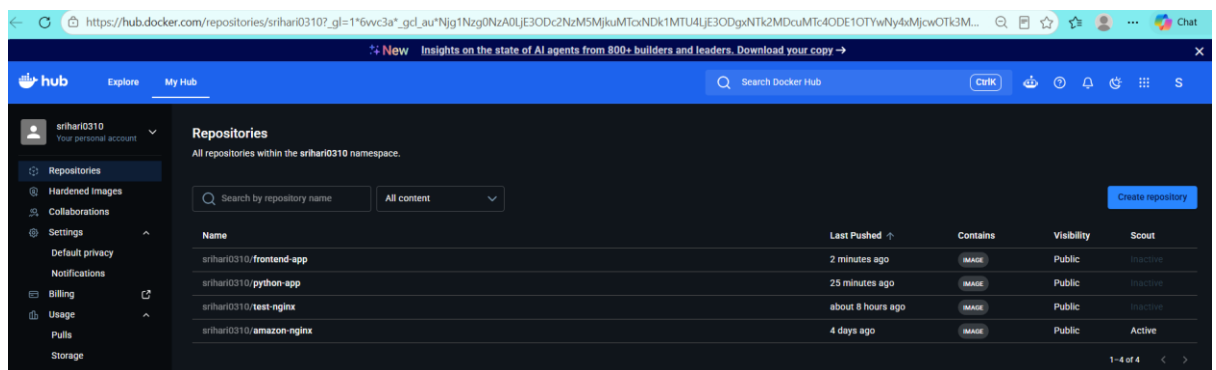
Push Image

- docker push srihari0310/frontend-app
Image uploaded to Docker Hub.

check Repository

verified:

srihari0310/frontend-app



From the **Java-based source code**, write a Dockerfile, build, run, and push the image to the Docker registry.

Move to Java Source Folder

- cd /source-code/docker-tasks/Java_based_source
ls

Open Dockerfile

- vi Dockerfile

3. Add Dockerfile Content (Multi-Stage Build)

Build stage

FROM maven:3.9.6-eclipse-temurin-17 AS build

WORKDIR /app

COPY . .

RUN mvn clean package -DskipTests

Runtime stage

FROM eclipse-temurin:17-jre

WORKDIR /app

COPY --from=build /app/target/*.jar app.jar

EXPOSE 8080

CMD ["java", "-jar", "app.jar"]

```
[root@ip-172-31-6-115 docker-tasks]# ls
Frontend_based_source  Java_based_source  NodeJs_based_source  README.md
[root@ip-172-31-6-115 docker-tasks]# cd Java_based_source/
[root@ip-172-31-6-115 Java_based_source]# ls
pom.xml  src
[root@ip-172-31-6-115 Java_based_source]# vi Dockerfile
[root@ip-172-31-6-115 Java_based_source]# cat Dockerfile
# Build stage
FROM maven:3.9.6-eclipse-temurin-17 AS build
WORKDIR /app
COPY . .
RUN mvn clean package -DskipTests

# Runtime stage
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar

EXPOSE 8080
CMD ["java", "-jar", "app.jar"]

[root@ip-172-31-6-115 Java_based_source]#
```

Build Image

- docker build -t srihari0310/java-app .

```
[root@ip-172-31-6-115 Java_based_source]# docker build -t srihari0310/java-app .
[+] Building 6.4s (7/14)
=> => transferring context: 2B
=> [Build 1/4] FROM docker.io/library/maven:3.9.6-eclipse-temurin-17@sha256:29a1650b1f3070e07c2b17f7b519b45eb47f65d9620e887eac45a0c5c0f939d4
=> resolve docker.io/library/maven:3.9.6-eclipse-temurin-17@sha256:29a1650b1f3070e07c2b17f7b519b45eb47f65d9620e887eac45a0c5c0f939d4
=> sha256:29a1650b1f3070e07c2b17f7b519b45eb47f65d9620e887eac45a0c5c0f939d4 1.21kB / 1.21kB
=> sha256:408d99a73e0dc1b14cfbc003f572920855da4003a266ebf841cf2c298a8d144 2.41kB / 2.41kB
=> sha256:b36805e902bf7f1eb3cd9d7f99f0cb391aacc9afe5653ba2c8279d2f0a4e2c23 7.91kB / 7.91kB
=> sha256:4a023cab5400feb5c1ab725beb8345ddb0e3200314004b56677a5eee2e8c86cf 30.44MB / 30.44MB
=> sha256:5e5d1bcc5440d3a24f4a620704b9e687b4163c6c872fcc8e812e200c9bbac58 17.46MB / 17.46MB
=> sha256:d59fd278c1b4ffe1727be6ccba42125e8f4db57e660e795ccce19889d2c776457 145.10MB / 145.10MB
=> sha256:c97285723537ab7921fca4d0b1c256e501adfaa8992d04637992075f4cea392 173B / 173B
=> sha256:a3b11f7aee4d96221d612f8e4ace9061aaebac6922f0274a1d204ab96be8 734B / 734B
=> extracting sha256:4a023cab5400feb5c1ab725beb8345ddb0e3200314004b56677a5eee2e8c86cf
=> sha256:67f99c2668af1d0ceded6d61dad057d7beffe543a8e8370d5c0d3fe8682e059a3 19.00MB / 19.00MB
=> sha256:45f480637770dbb740623c0c7f827d44bbacc9204faf3d07b4135d54a2bee043 9.48MB / 9.48MB
=> sha256:58c3491a14ebc973960cbda9f2eea22537785a3cb07e81487bd1115bad4a8278 852B / 852B
=> sha256:4712dfa85971124b8c3507fbb4c02441e5754f1517162926e600afd3fc5404b 358B / 358B
=> sha256:fc06d68d71ba375a9ceed165c0c04b09b1a8cc19314688663b005660fc25013 156B / 156B
=> extracting sha256:5e5d1bcc5440d3a24f4a620704b9e687b4163c6c872fcc8e812e200c9bbac58
=> [Internal] load build context
=> => transferring context: 1.89MB
=> [stage-1 1/3] FROM docker.io/library/eclipse-temurin:17-jre@sha256:13cc28a6cc72a38ce1f00c906be3580c1a3e604b8984d694f369a96742abc93b
=> resolve docker.io/library/eclipse-temurin:17-jre@sha256:13cc28a6cc72a38ce1f00c906be3580c1a3e604b8984d694f369a96742abc93b
=> sha256:13cc28a6cc72a38ce1f00c906be3580c1a3e604b8984d694f369a96742abc93b 7.22kB / 7.22kB
=> sha256:c984fca94ab42452795a0bee45a35a92318390a673d890b8216e9677ac21aa37 2.13kB / 2.13kB
=> sha256:9a3b1de629c51deedfee94b5b33721d99e4bf1f0f208bead2879ec457b006cbea 8.99kB / 8.99kB
=> sha256:06e9d71331fb2cb20a4f6c8064e0f84b284bb69a42c7c57b1c962bd4a4cdee76 41.57MB / 41.57MB
=> sha256:f3db1c94076639b09d8a60e47c66fa95020788ecfab5bec91a4a77d4ced1c 393B / 393B
=> sha256:1a05c661b3f4d3ba3e791ae4405a9d8d7c4be76580b0c3978c9e9d1eea51e60 20.13MB / 20.13MB
=> sha256:409c221e1962921dd400761b763523ed5171be79f0c000b7401bf5e2326b16 47.52MB / 47.52MB
=> sha256:074b156e16db041e40e80e64efb5d1c599275c1b0a984c8d55a5fc3ac3951b 2.46kB / 2.46kB
=> sha256:d12223f377f052485212a69a1db48316ea7dd9bfe606c4a6cf8d9d92bba53e5 158B / 158B
```

Run Container

- docker run -d -p 8086:8080 --name javaapp srihari0310/java-app

```
[root@ip-172-31-6-115 Java_based_source]# docker run -d -p 8086:8080 --name javaapp srihari0310/java-app
97f7e866fa399be61b684196bdadc7c06192615528038422efb12fe0e9629c98
[root@ip-172-31-6-115 Java_based_source]#
```



Login to Docker Hub

- docker login

Push Image

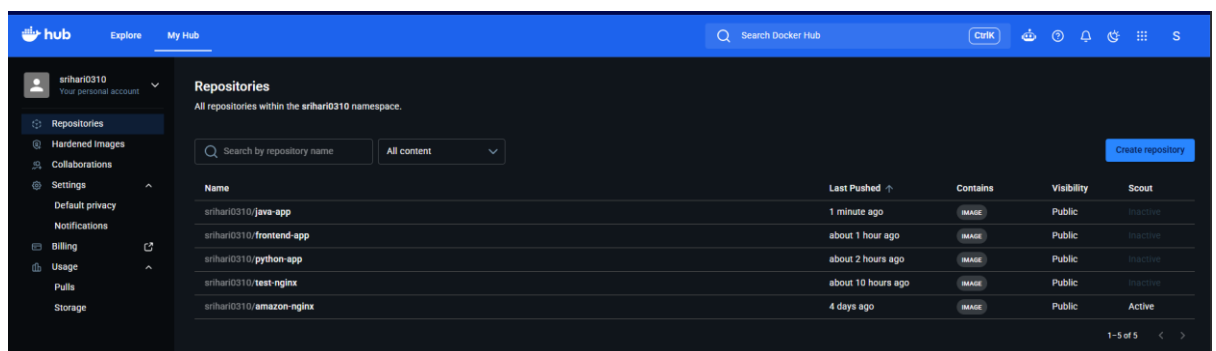
- docker push srihari0310/java-app

```
[root@ip-172-31-6-115 Java_based_source]# docker push srihari0310/java-app
Using default tag: latest
The push refers to repository [docker.io/srihari0310/java-app]
0d9af18dfc61: Pushed
1a1073964db3: Pushed
281852bcff0e: Mounted from library/eclipse-temurin
1d4ad11192b0: Mounted from library/eclipse-temurin
41cb3b727c6b: Mounted from library/eclipse-temurin
9c9c5161ed13: Mounted from library/eclipse-temurin
168f4b64baaa: Mounted from library/eclipse-temurin
b577adbc0b58: Mounted from library/eclipse-temurin
latest: digest: sha256:4d4b55b774887835ab5a2750e941a2eb588fdb250f14ce91f475b93a133ec25d size: 1993
[root@ip-172-31-6-115 Java_based_source]#
```

Check Repository

confirmed:

srihari0310/java-app



From the **Node.js-based source code**, write a Dockerfile, build with tag v1, run, and push it to the Docker registry.

Go to Node.js Project Directory

- `cd /source-code/docker-tasks/NodeJs_based_source`
`ls`

Create Dockerfile

- `vi Dockerfile`

Add Dockerfile Content

`FROM node:18-alpine`

`WORKDIR /app`

`COPY package*.json ./`

`RUN npm install`

`COPY . .`

`EXPOSE 3000`

`CMD ["npm","start"]`

Build Image

- `docker build -t srihari0310/node-app:v1 .`

```
[root@ip-172-31-6-115 docker-tasks]# ls
frontend_based_source  Java_based_source  NodeJs_based_source  README.md
[root@ip-172-31-6-115 docker-tasks]# cd NodeJs_based_source/
[root@ip-172-31-6-115 NodeJs_based_source]# vi Dockerfile
[root@ip-172-31-6-115 NodeJs_based_source]# cat Dockerfile
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm","start"]

[root@ip-172-31-6-115 NodeJs_based_source]# docker build -t srihari0310/node-app:v1 .
[+] Building 9.8s (8/10)
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 21kB
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d662b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d662b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> => sha256:1e844c89ee8e5c026e540ab0c6d4b6491c96eda01337f439ba47f0c2e702d6e9 1.20kB / 1.20kB
=> => sha256:8d6421d662b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e 7.67kB / 7.67kB
=> => sha256:929b04d7c782f04f615cf785498fed452b6569f87c73ff66ead553a7554f0006 1.72kB / 1.72kB
=> => sha256:ee77c6cd7c1886ecc902ad6cedf3a8ec1ea27d1fb96162bf03dd3710839b8da 6.18kB / 6.18kB
=> => sha256:f18232174bc91741fd3da96d85011092101a032a93a388b79e99e69c2d5c870 3.64MB / 3.64MB
```

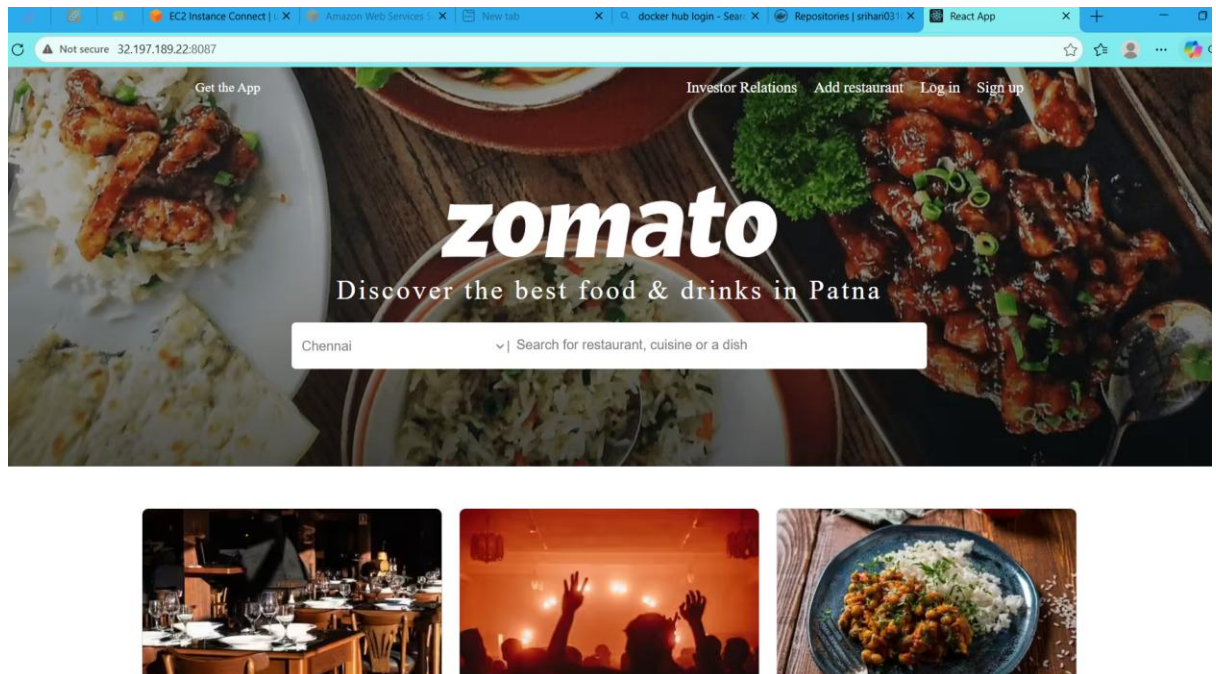
Run Container

- `docker run -d -p 8087:3000 --name nodeapp srihari0310/node-app:v1`

```
[root@ip-172-31-6-115 ~]# ls
source-code
[root@ip-172-31-6-115 ~]# cd source-code/
[root@ip-172-31-6-115 source-code]# ls
docker-tasks
[root@ip-172-31-6-115 source-code]# cd docker-tasks/
[root@ip-172-31-6-115 docker-tasks]# ls
Frontend_based_source  Java_based_source  NodeJs_based_source  README.md
[root@ip-172-31-6-115 docker-tasks]# cd NodeJs_based_source/
[root@ip-172-31-6-115 NodeJs_based_source]# vi Dockerfile
[root@ip-172-31-6-115 NodeJs_based_source]# docker build -t srihari0310/node-app:v1 .
[*] Building 0.4s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 216B
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> [internal] load build context
=> => transferring context: 6.04kB
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY package*.json ./
=> CACHED [4/5] RUN npm install
=> CACHED [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => writing image sha256:d50f594211f1f03855a0376da446c71a23168dda200e8ca02d2985ad319dc36b
=> naming to docker.io/srihari0310/node-app:v1
[root@ip-172-31-6-115 NodeJs_based_source]# docker run -d -p 8087:3000 --name nodeapp srihari0310/node-app:v1
-bash: docker: command not found
[root@ip-172-31-6-115 NodeJs_based_source]# docker run -d -p 8087:3000 --name nodeapp srihari0310/node-app:v1
```

Open in Browser

http://<EC2-Public-IP>:8087

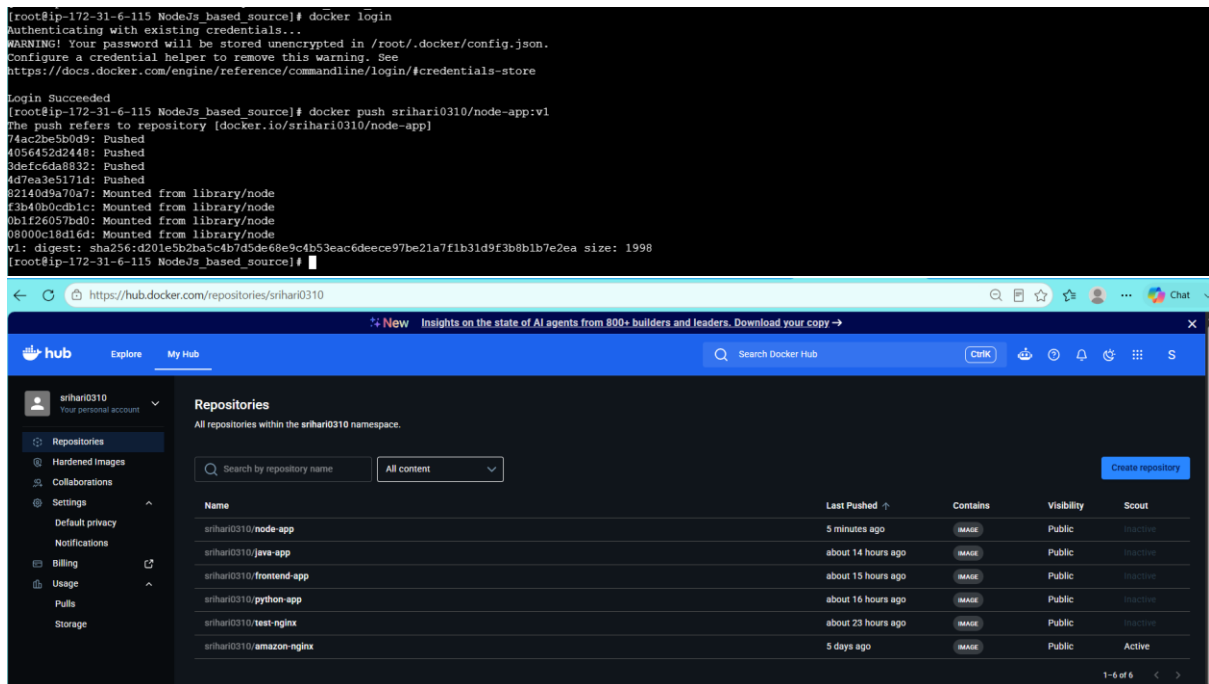


Login to Docker Hub

- docker login

Push Image

- docker push srihari0310/node-app:v1



- Write a **docker-compose file** to set up WordPress with a MySQL database.

Create Working Directory

- mkdir wordpress-compose
- cd wordpress-compose

Create docker-compose.yml

- vi docker-compose.yml

Add Docker Compose Configuration

version: '3.8'

services:

db:

image: mysql:8

container_name: wordpress-db

restart: always

environment:

MYSQL_DATABASE: wordpress

MYSQL_USER: wpuser

MYSQL_PASSWORD: wppass

MYSQL_ROOT_PASSWORD: rootpass

volumes:

- db_data:/var/lib/mysql

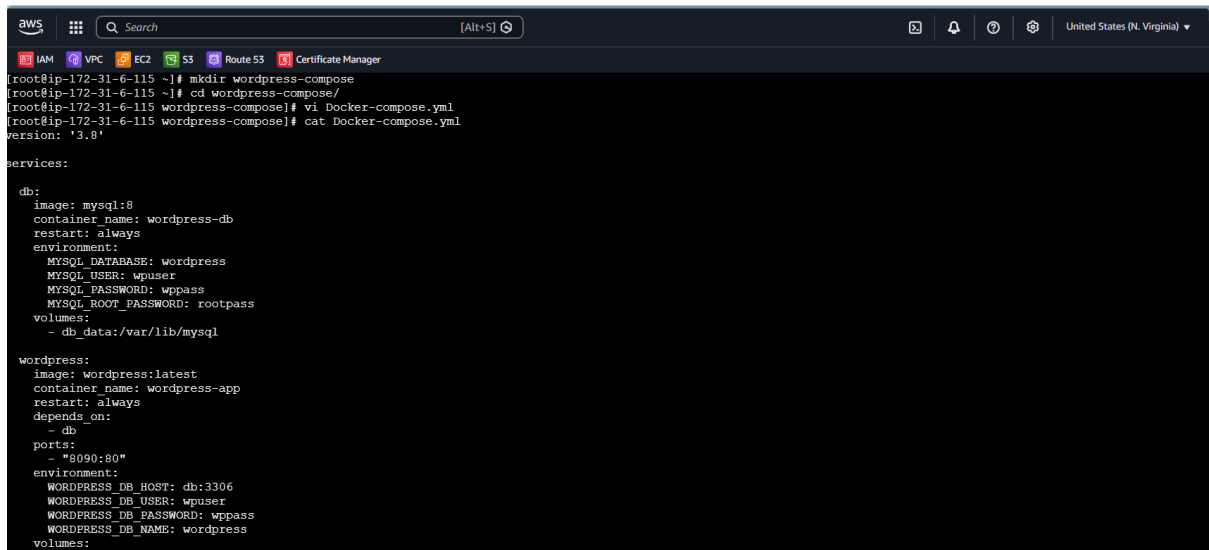
wordpress:

image: wordpress:latest

container_name: wordpress-app

```
restart: always
depends_on:
  - db
ports:
  - "8090:80"
environment:
  WORDPRESS_DB_HOST: db:3306
  WORDPRESS_DB_USER: wpuser
  WORDPRESS_DB_PASSWORD: wppass
  WORDPRESS_DB_NAME: wordpress
volumes:
  - wp_data:/var/www/html
```

```
volumes:
db_data:
wp_data:
```



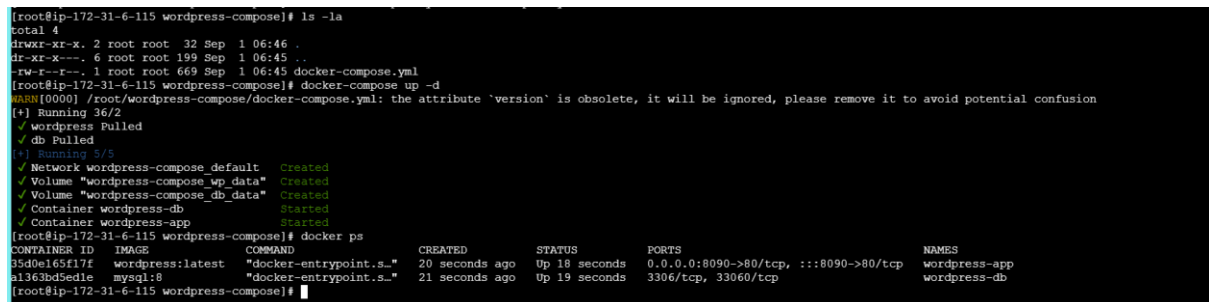
```
aws
[Alt+S]
IAM VPC EC2 S3 Route 53 Certificate Manager
[root@ip-172-31-6-115 ~]# mkdir wordpress-compose
[root@ip-172-31-6-115 ~]# cd wordpress-compose/
[root@ip-172-31-6-115 wordpress-compose]# vi Docker-compose.yml
[root@ip-172-31-6-115 wordpress-compose]# cat Docker-compose.yml
version: '3.8'

services:
  db:
    image: mysql:8
    container_name: wordpress-db
    restart: always
    environment:
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
      MYSQL_ROOT_PASSWORD: rootpass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    container_name: wordpress-app
    restart: always
    depends_on:
      - db
    ports:
      - "8090:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
    volumes:
```

Launch Containers

➤ `docker-compose up -d`



```
[root@ip-172-31-6-115 wordpress-compose]# ls -la
total 4
drwxr-xr-x. 2 root root 32 Sep  1 06:46 .
dr-xr-x---. 6 root root 199 Sep  1 06:45 ..
-rw-r--r--. 1 root root 669 Sep  1 06:45 docker-compose.yml
[root@ip-172-31-6-115 wordpress-compose]# docker-compose up -d
WARN[0000] /root/wordpress-compose/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[*] Running 36/2
✓ wordpress Pulled
✓ db Pulled
[*] Running 5/5
✓ Network wordpress-compose default Created
✓ Volume "wordpress-compose_wp_data" Created
✓ Volume "wordpress-compose_db_data" Created
✓ Container wordpress-db Started
✓ Container wordpress-app Started
[root@ip-172-31-6-115 wordpress-compose]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
35d0e165f17f   wordpress:latest "docker-entrypoint.s..." 20 seconds ago Up 18 seconds 0.0.0.0:8090->80/tcp, :::8090->80/tcp  wordpress-app
a1363b45ed1e   mysql:8       "docker-entrypoint.s..." 21 seconds ago Up 19 seconds  3306/tcp, 33060/tcp                wordpress-db
[root@ip-172-31-6-115 wordpress-compose]#
```

Open WordPress Setup Page

In browser, opened:

<http://<EC2-Public-IP>:8090/wp-admin/install.php>

